



Design of SPI-Slave with Register Map

Architecture, Finite-State-Machine, and Functional Verification Report

Report by: Youssef Bassem Lamey

Submitted to: Eng. Fatma Ali

ADI Summer Internship 2026 — Digital Design and Verification Team

July 2026

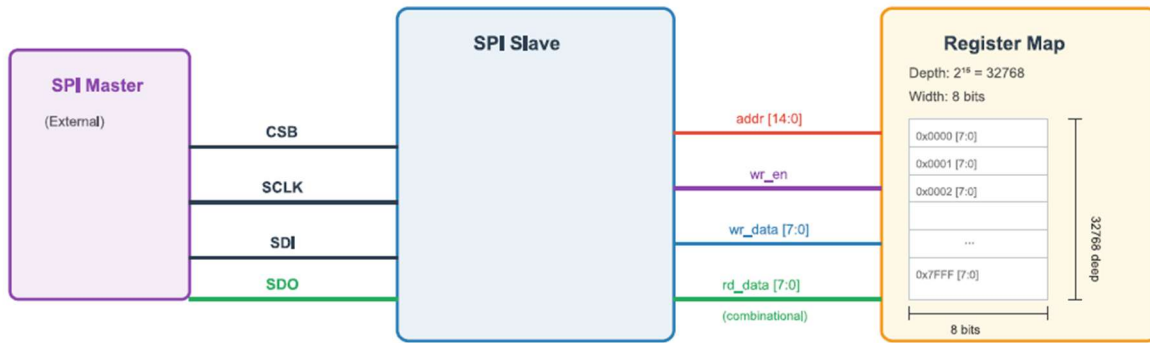
Contents

1. Introduction	3
2. Architecture	3
2.1 Transaction Framing	4
2.2 Clock Gating — IGC Cell	4
2.3 Register Map	5
3. Features	5
4. Finite State Machine	6
5. Test Cases	7
5.1 Test Case 1 — 4 Write Bursts	7
5.2 Test Case 2 — 2 Reads	8
6. Conclusion	8

1. Introduction

This report documents the design of a 4-wire SPI slave interface (CSB, SCLK, SDI, SDO) that provides read and write access to an internal $32,768 \times 8$ register map. The interface, its transaction framing, and its verification plan follow the design frame prepared for this assignment; this document describes the resulting RTL architecture, its finite-state machine, and the functional test cases used to verify single- and multi-byte write bursts and single-address reads.

2. Architecture



The design is organized as a top-level wrapper, `spi_slave_wrapper`, that instantiates two blocks:

- **spi_slave** — the protocol engine. It deserializes SDI into a command bit, a 15-bit address and 8-bit data bytes, drives the FSM, and serializes register-map read data back out on SDO.
- **reg_map** — the storage array (DEPTH = $2^{15} = 32,768$ locations $\times$ 8 bits). Writes are synchronous to SCLK and combinationaly readable on `rd_data`.

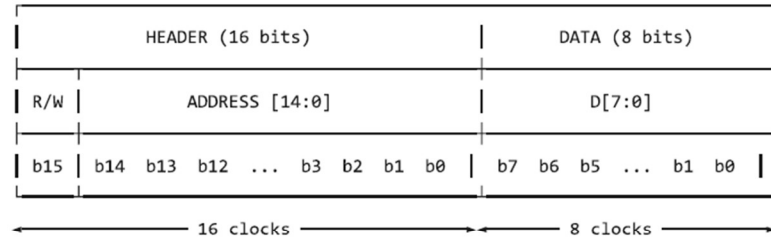
Port list of the `spi_slave` / `spi_slave_wrapper` interface:

Signal	Dir.	Width	Description
SCLK	In	1	Serial clock from the SPI master.
rst_n	In	1	Asynchronous, active-low reset.
CSB	In	1	Active-low chip select; frames a transaction.
SDI	In	1	Serial data in (MSB-first: R/W, address, write data).
SDO	Out	1	Serial data out (read data, MSB-first).

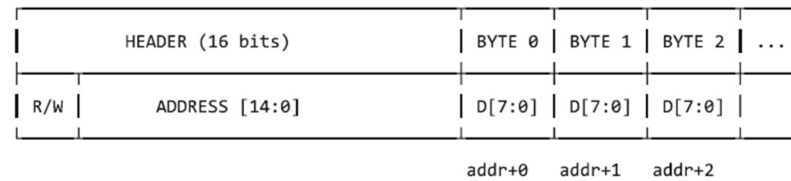
2.1 Transaction Framing

Every transaction begins with a 16-bit header — a single R/W bit followed by a 15-bit address — shifted in MSB-first, followed by one or more 8-bit data bytes:

Single Byte Access (24 bits):



Multi-Byte Burst (auto-increment):



Field	Bits	Description
R/W	[15]	0 = Write (master drives data on SDI); 1 = Read (slave drives data on SDO).
ADDRESS	[14:0]	Starting register address; auto-increments by 1 after every complete write byte.
DATA	8 per byte	Register data, MSB-first. One or more bytes may follow the header.

A single register access is therefore 24 clocks (16 header + 8 data). Additional back-to-back data bytes within the same CSB assertion form a burst: `reg_map` auto-increments its internal write offset after every completed byte, so consecutive bytes land in consecutive addresses without the master having to resend the header.

2.2 Clock Gating — IGC Cell

Internally, `spi_slave` does not run its counters and shift registers directly from SCLK. Instead it generates a gated internal clock, CLK, through an integrated-clock-gating (IGC) cell:

- A level-sensitive latch (`en_latch`) is transparent while SCLK is low and captures $\sim$ CSB at that time.
- The gated clock is then formed combinatorially as $\text{CLK} = \text{SCLK} \ \& \ \text{en_latch}$.

Because the enable is captured in a latch that is only transparent during the SCLK-low phase, any change on CSB while SCLK is high cannot ripple through and create a runt or glitched pulse on CLK — the new enable value is held off until the next SCLK-low window, so every pulse that does reach CLK is a full, clean SCLK pulse. This is the standard latch-based ICG (Integrated Clock Gating) structure used in ASIC design specifically to gate a clock safely, as opposed to gating it with a plain combinational AND of SCLK

and CSB, which would be glitch-prone. All of the address/data counters, `wr_en`, and the write-data/address registers are clocked from this gated CLK rather than from raw SCLK.

2.3 Register Map

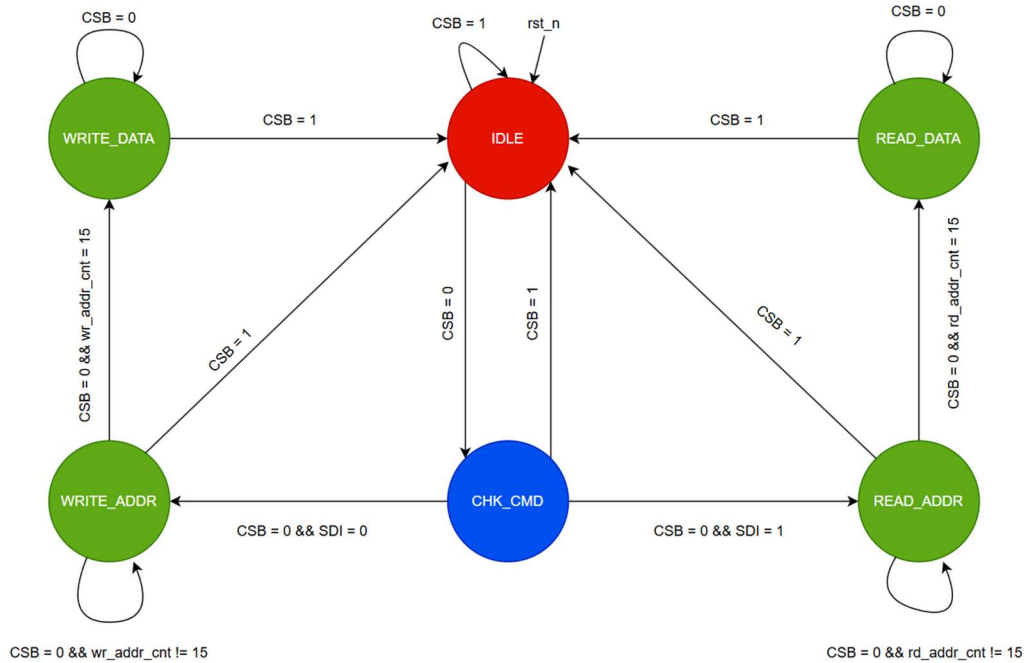
`reg_map` implements the storage array as `ram[0:32767]`, each entry 8 bits wide. On every asserted `wr_en` pulse it writes `wr_data` into `ram[addr + offset]` and increments `offset` by one, which is the mechanism that realizes burst auto-increment. `rd_data` is a continuous combinational read, `ram[addr]`, satisfying the half-clock read-turnaround requirement (the slave has only half a SCLK period after the last address bit to present the first read bit on SDO). An asynchronous active-low reset clears the entire array and the offset counter.

3. Features

- 4-wire SPI slave (CSB, SCLK, SDI, SDO), MSB-first serial framing.
- 15-bit address bus — 32,768 × 8-bit internal register map.
- 16-bit header (1 R/W bit + 15-bit address) followed by any number of 8-bit data bytes.
- Single-byte access and multi-byte burst access share the same framing — burst mode is inferred purely from continued CLK activity after the first data byte, with no dedicated burst bit.
- Write-side auto-increment: the target address advances by one for every completed data byte in `reg_map`.
- Combinational read path: read data is valid on SDO within half a CLK period of the last address bit, meeting the read-turnaround requirement.
- Latch-based IGC (integrated clock gating) cell derives the internal CLK from SCLK and CSB, avoiding combinational glitches on the gated clock.
- Asynchronous, active-low reset clears the full register array and all internal counters/registers.
- CSB framing with defined `t_lead` / `t_lag` margins around the SCLK burst (`t_lag` assumed equal to `t_lead`).

4. Finite State Machine

The protocol engine is a 6-state Moore-style machine (ps/ns) clocked on the posedge of SCLK, with an asynchronous active-low reset returning it to IDLE:



State	Function	Next-state condition
IDLE	Reset / waiting state. No transaction in progress.	CSB = 0 → CHK_CMD
CHK_CMD	Samples the first SDI bit after CSB asserts to decide the transaction type.	SDI = 0 → WRITE_ADDR; SDI = 1 → READ_ADDR
WRITE_ADDR	Shifts in the 15-bit register address (wr_addr_cnt counts 0–14).	wr_addr_cnt = 15 → WRITE_DATA
WRITE_DATA	Shifts in 8-bit data bytes; pulses wr_en once per completed byte. Stays in this state — and keeps consuming bytes — for as long as CSB remains low, which is how burst writes are formed.	CSB = 1 → IDLE
READ_ADDR	Shifts in the 15-bit register address for a read (rd_addr_cnt counts 0–14).	rd_addr_cnt = 15 → READ_DATA
READ_DATA	Drives rd_data onto SDO, MSB-first, combinationaly.	CSB = 1 → IDLE

CSB is the master control signal for the FSM: from any of the four active states, CSB = 1 forces an immediate return to IDLE (ending the transaction), while CSB = 0 keeps the machine progressing through the header and data phases. CHK_CMD is a single-cycle decode state that reads SDI once to route the transaction to the write or read branch.

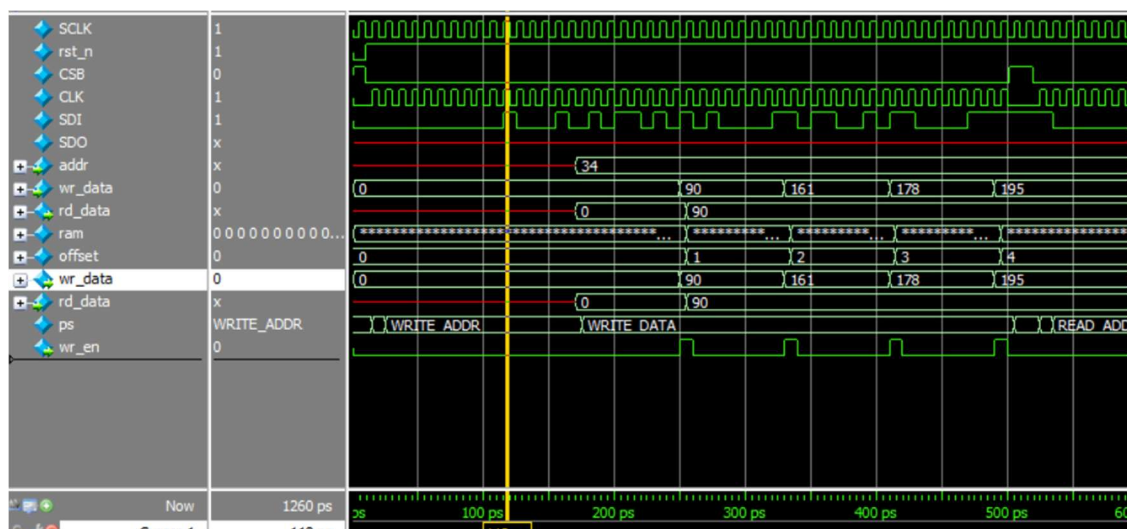
5. Test Cases

The testbench exercises the design with two functional scenarios: a 4-byte write burst starting at address 34, and two single-address reads at addresses 36 and 37 that read back bytes written during the burst.

5.1 Test Case 1 — 4 Write Bursts

A single CSB assertion sends the header (W = 0, address = 34) followed by four consecutive 8-bit data bytes. reg_map's offset increments after each byte, so the four bytes land in four consecutive addresses without re-sending the header:

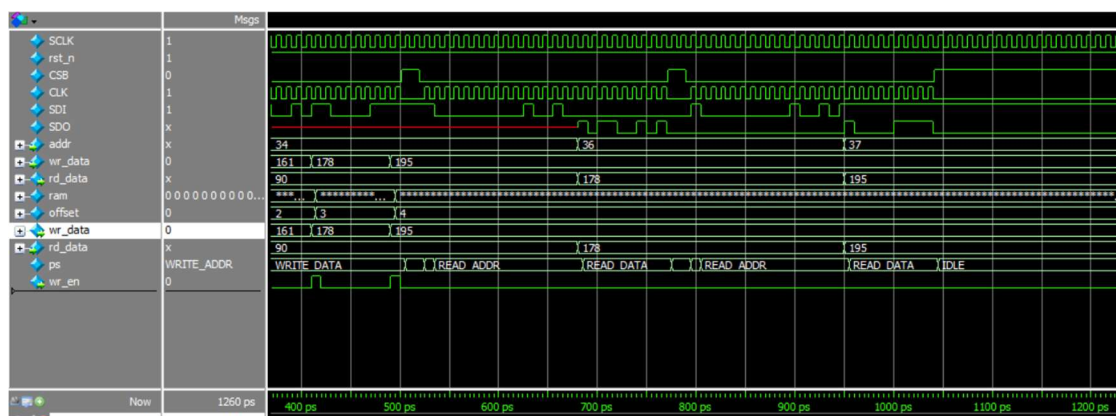
Byte #	SDI Data	Target Address (addr + offset)	reg_map offset after write
1	0x5A (90)	34	1
2	0xA1 (161)	35	2
3	0xB2 (178)	36	3
4	0xC3 (195)	37	4



5.2 Test Case 2 — 2 Reads

After the write burst, two independent read transactions are issued — each with its own header (R = 1) — to addresses 36 and 37, i.e. the third and fourth locations written in Test Case 1:

Read #	Address	Expected Data	Captured on SDO	Result
1	36	0xB2 (178)	178	Match
2	37	0xC3 (195)	195	Match



6. Conclusion

The waveform evidence in Test Cases 1 and 2 confirms that the `spi_slave / reg_map` pair correctly implements the specified 4-wire protocol: the header is decoded once per transaction, write bursts auto-increment through `reg_map`'s offset counter without re-sending the address, and single-address reads correctly return the byte previously written at that location. The latch-based IGC cell keeps the internal CLK glitch-free while gating it with CSB, and the FSM cleanly returns to IDLE on CSB de-assertion between transactions, matching the `t_lead / t_lag` framing defined for the interface.